Digital Alarm Clock

A Project Report

---

1. Objective

The objective of this project is to design and implement a Digital Alarm Clock that not only displays time and alarms but also incorporates interactive and engaging features such as multi-alarm support, customizable snooze durations, and an innovative alarm deactivation system to ensure active user participation when dismissing alarms.

1.1 Goals

The goal is to create a user-friendly digital alarm clock that:

1. **Multi-Alarm Management:** Allows users to set and manage multiple alarms easily through the keypad.
2. **Flexible Snooze:** Provides customizable snooze options (1, 2, or 3 minutes) based on user preference.
3. **Interactive Dismissal:** Enhances alarm dismissal by requiring users to input a secret key, ensuring the user is awake and alert.
4. **Full Peripheral Integration:** Integrates a loud buzzer, a water-spray motor, a clear LCD display, and a responsive 4×4 keypad for effective interaction.

2. Required Components

Component	Cost (BDT)
Microcontroller (AT89C52)	5,000
Keypad (4×4)	Built-in
Jumper Wires	200
Buzzer	Built-in
LED	Built-in
LCD (LM016L)	Built-in
Motor (water pump)	1,500
MOSFET	50
TOTAL	**6,750 BDT**

3. Circuit Diagram

3.1 Proteus Simulation Layout

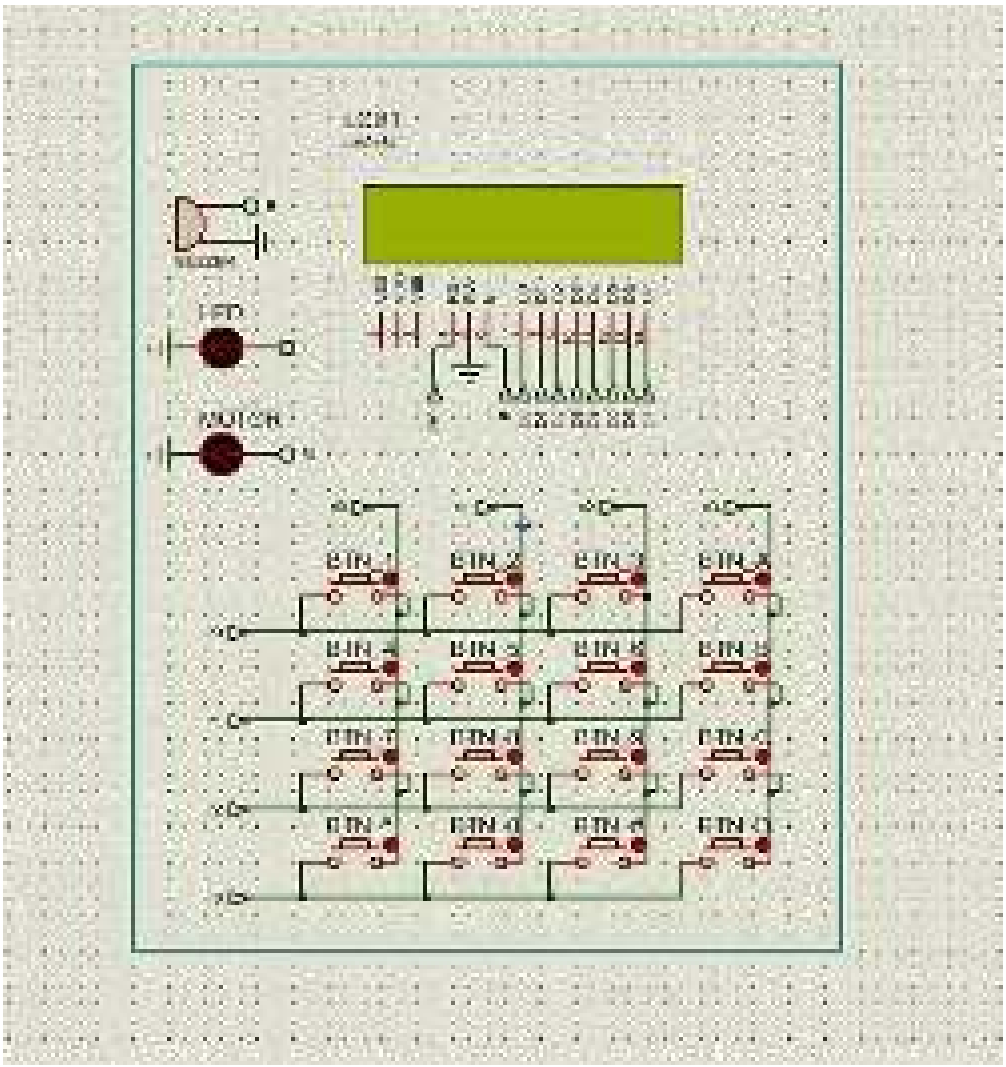


Fig 3.1 — Proteus simulation: LCD, Buzzer, LED, Motor, and 4×4 Keypad connected to AT89C52

3.2 Schematic Diagram

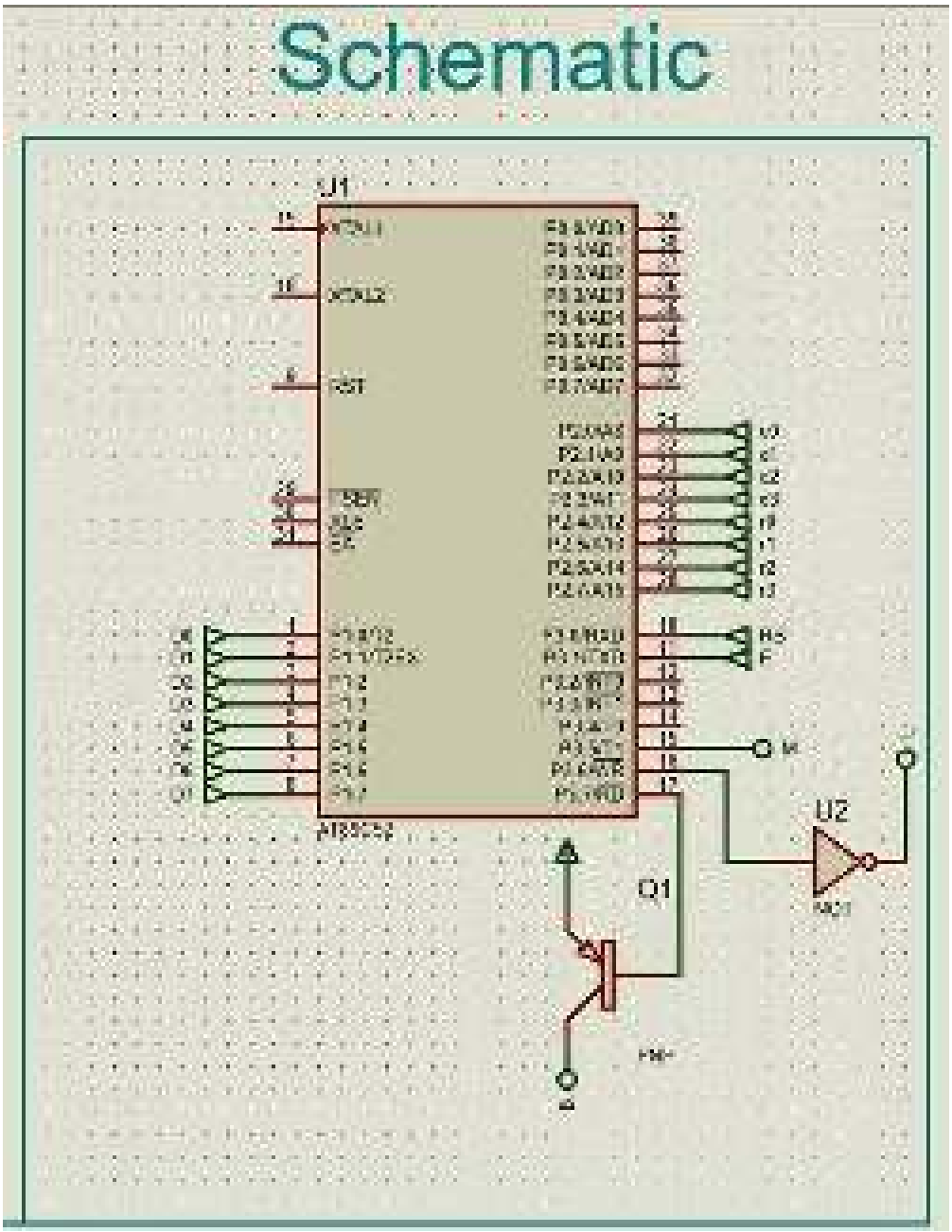


Fig 3.2 — Full schematic: AT89C52 microcontroller pin assignments, PNP transistor motor driver, and NOT gate

3.3 Port & Pin Assignment

Port / Pin	Connected To	Description
Port 1 (P1.0–P1.7)	**LCD (LM016L)**	Data bus — sends 8-bit data/commands to the LCD
Port 2 (P2.0–P2.3)	**Keypad Rows (r0–r3)**	Lower nibble used for keypad row scanning
Port 2 (P2.4–P2.7)	**Keypad Cols (c0–c3)**	Upper nibble used for keypad column scanning

P3.0 (RS)	**LCD RS pin**	Register Select — distinguishes command vs data
P3.1 (Enable)	**LCD Enable pin**	Strobe pin — latches data into LCD on falling edge
P3.5	**Motor (via MOSFET)**	Water-spray motor — activated in Alarm Resolve Mode
P3.6	**Alarm Indicator LED**	Illuminates when alarm is active (ALRMSET)
P3.7	**Buzzer**	Sounds audible alert in Alarm Resolve Mode

4. Features

4.1 Clock Display Mode (Mode 1 — Default)

- The clock continuously displays current time (HH:MM:SS) on the LCD, updating every second.
- The display remains active at all times, ensuring the user can view the current time at a glance.
- When an alarm is set, an **'A'** indicator appears in the top-right corner of the LCD.

4.2 Edit Time Mode (Mode 2 — triggered by 'D' key)

- Allows the user to manually set hours, minutes, and seconds via the keypad.
- Validates all inputs (H < 24, M < 60, S < 60) before accepting.
- After saving, the mode automatically returns to Clock Display Mode.

4.3 Alarm Set Mode (Mode 3 — triggered by '*' key)

- Allows the user to set an alarm by entering hours and minutes (HH:MM).
- Validates inputs (H < 24, M < 60) and confirms with a 0/1 flag.
- After setting, returns to Clock Display Mode with the 'A' indicator active.

4.4 Alarm Resolve Mode (Mode 4 — auto-triggered on alarm match)

- **Buzzer:** Activates the buzzer (P3.7 set LOW) to produce a loud auditory alert.
- **Motor:** Activates the water-spray motor (P3.5 HIGH) via a MOSFET/PNP driver, providing a physical stimulus.
- **Secret Key:** Prompts the user to enter a secret key on the LCD. If incorrect, the alarm continues ringing.
- **Stop / Snooze:** On correct key entry, the user presses 0 to stop the alarm, or 1/2/3 to snooze for 1, 2, or 3 minutes respectively.

5. System Flowchart

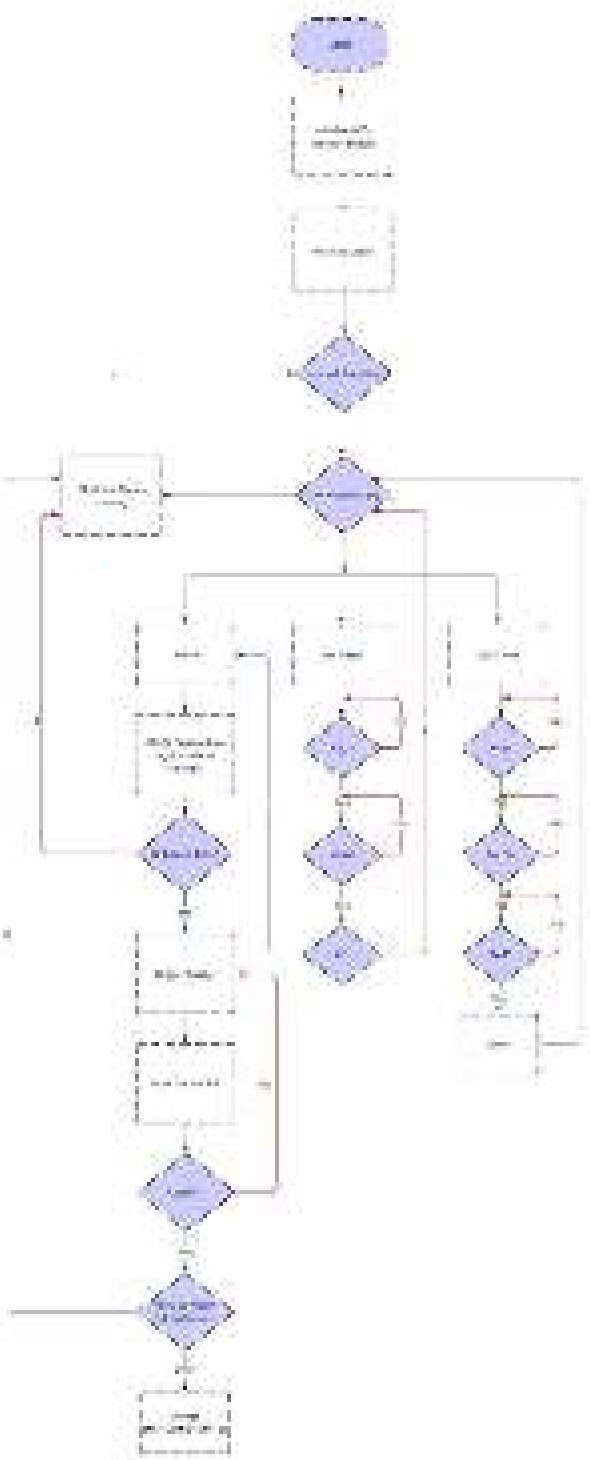


Fig 5.1 — Complete system flowchart: initialization → keypad scan → decision → Clock / Set Alarm / Edit Clock / Alarm Resolve branches

The flowchart illustrates the four operating branches. After initialization, the system enters a continuous keypad scanning loop. The Decision Making node routes execution to the

appropriate mode. The Alarm branch checks the current time against alarm settings each loop; when matched, Trigger Buzzer is called and the user must enter the secret key before reaching the Stop/Snooze decision.

6. Working Principle

6.1 Initialization Phase

The program begins by configuring all peripherals and zeroing all time variables:

Module	Initialization Action
LCD	Configured with 38H (8-bit, 2-line), 0CH (display on), 06H (entry mode), 01H (clear)
Keypad	PORT 2 set to 0FFH (all lines HIGH, ready for scanning)
Timer 1	Set in Mode 1 (16-bit), reload values 0DBH/0F0H for 10 ms interrupt intervals
Variables	REALHOUR, REALMINT, REALSCND, ALRMHOUR, ALRMMINT, ISALRM, MODE all set to 0
Buzzer	SETB P3.7 (HIGH = off, active-low configuration)
Motor	CLR P3.5 (LOW = off)

6.2 Main Loop Operation

The code enters an infinite loop (MAINLOOP) performing three continuous tasks:

5. Scans the keypad for any user input and stores the detected key in KEYMEM.
6. Updates the LCD display with the current time (REALHOUR:REALMINT:REALSCND).
7. Compares **REALMINT / REALHOUR** against **ALRMMINT / ALRMHOUR** — if both match and ISALRM ≠ 0, MODE is set to 04H (Alarm Resolve).

6.3 Mode Switching

Mode	Name	Trigger	Description
01H	**Clock Display**	Default / on save	Shows HH:MM:SS in real time. 'A' shown if alarm active.
02H	**Edit Time**	'D' key pressed	User enters H, M, S. Validates ranges. Returns to Mode 1.
03H	**Alarm Set**	'*' key pressed	User enters alarm HH:MM and enables (0/1). Returns to Mode 1.
04H	**Alarm Resolve**	Clock matches alarm time	Buzzer + motor active. User must enter secret key, then

			stop/snooze.

6.4 Timer 1 Interrupt (TIMER1ISR) — 10 ms intervals

Timer 1 is configured to interrupt every 10 ms. On each interrupt, the following cascade of counter updates takes place:

Counter	Overflow Condition	Action on Overflow
REALMSCND	Reaches 100 (= 1 second)	Reset to 0; increment REALSCND by 1
REALSCND	Reaches 60 (= 1 minute)	Reset to 0; increment REALMINT by 1
REALMINT	Reaches 60 (= 1 hour)	Reset to 0; increment REALHOUR by 1
REALHOUR	Reaches 24 (= 1 day)	Reset to 0 (24-hour wrap-around)

The Timer 1 reload values are **TH1 = 0DCH, TL1 = 000H** on each re-entry, maintaining precise 10 ms intervals. While in Alarm Resolve Mode (MODE = 04H), the ISR also keeps the buzzer active (**CLR BUZZER**) during each tick.

6.5 Keypad Scanning & Input Handling

The 4×4 keypad is scanned using a row-column method on **Port 2**. Columns are driven HIGH (0F0H) first to detect active rows, then rows are driven HIGH (00FH) to detect active columns:

- **Column scan:** PORT2 ← 0F0H, read upper nibble → call KEYPAD_IDX → store column index in R0
- **Row scan:** PORT2 ← 00FH, read lower nibble → call KEYPAD_IDX → store row index in R1
- **Key lookup:** Index = R0×4 + R1, look up KEYPADBUF table → store ASCII char in KEYMEM
- **Debounce:** KEY_TAKE_ONE_DIGIT scans twice with a DELAY between reads and only accepts stable, consistent key presses

Key	Action
0–9	Digit input for time, alarm, snooze, and secret key entry
*** (star)**	Switch to Mode 3 (Alarm Set Mode)
D	Switch to Mode 2 (Edit Time Mode)
0 (in alarm)	Stop alarm — clears ISALRM and ISSNZ
1/2/3 (in alarm)	Snooze for 1/2/3 minutes — adds to ALRMMINT

7. Hardware Implementation

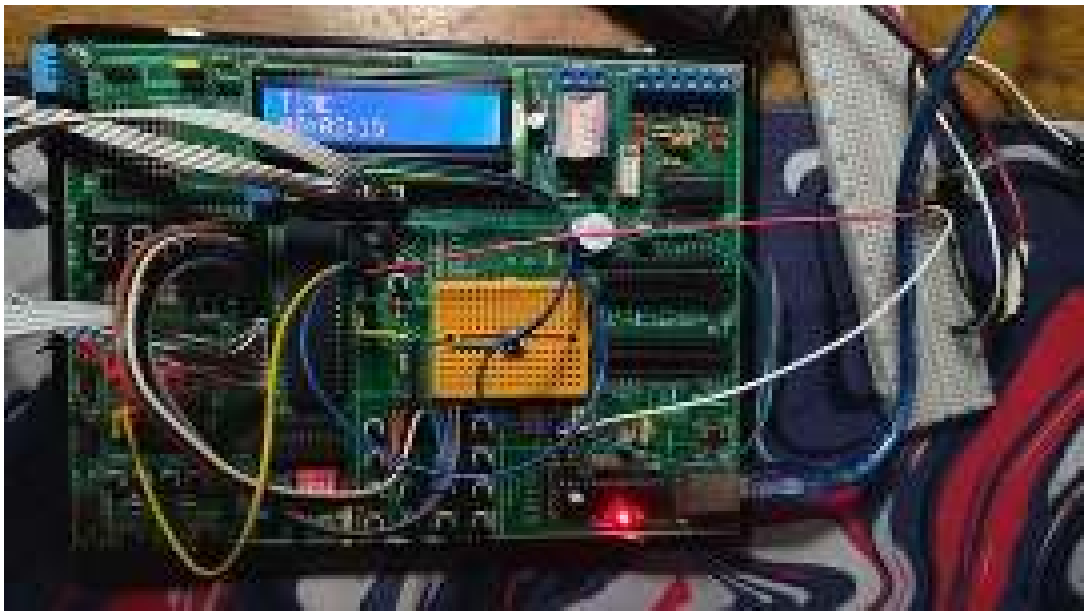


Fig 7.1 — Physical implementation on AT89C52 board showing LCD displaying TIME 00:02:15, keypad, and alarm indicator LED (red)

The system was implemented on a dedicated AT89C52 development board. The LCD clearly displays the clock in TIME HH:MM:SS format. The 4×4 matrix keypad is visible at the bottom. The red LED (alarm indicator) is visible at the right. The motor driver (PNP + MOSFET) is connected to P3.5 for the water-spray mechanism.

8. Problems Faced

8.1 Software Issues

Problem	Description
Concurrent Input & Timer	Difficult to accept keypad input for Alarm Set Mode while Timer 1 was running and continuously refreshing the LCD display.
LCD Refresh Glitch	Real-time display was not updating as expected — residual/garbage characters appeared on the LCD.
Keypad Debouncing	Keypad produced double inputs due to contact bounce; resolved using double-scan debouncing in KEY_TAKE_ONE_DIGIT.
Multiple Alarms	Second alarm was not triggering properly — implementing multiple alarms would require enhanced memory management and interrupt prioritization.

LED Puzzle Complexity	The LED-pattern puzzle for dismissal was too complex to implement reliably; replaced with a simpler secret key input.

8.2 Hardware Issues

Problem	Description
Insufficient Motor Power	The microcontroller board could not supply enough current on P3.5 to drive the motor; a MOSFET/PNP driver stage was required.
Faulty Port 1	Port 1 of the board was defective; Port 0 was used instead for the LCD data bus in the physical build.
Loose Burning Switch	The programming (burning) switch had a loose connection, causing multiple failed code-upload attempts before a successful flash.

9. Conclusion

9.1 Delivered Outcomes

8. **Real-Time Clock:** Accurately tracks and displays HH:MM:SS using Timer 1 interrupts. Users can manually edit the time via the keypad.
9. **Alarm System:** Triggers both auditory (buzzer) and physical (water-spray motor) stimuli for effective wake-up alerts. Snooze of 1, 2, or 3 minutes is supported.
10. **Interactive Dismissal:** The secret key input ensures user engagement. The alarm continues until the correct key is provided.

9.2 Areas for Improvement

- **Multiple Alarms:** Requires enhanced memory management and interrupt prioritization beyond the current single-alarm implementation.
- **Debounce Circuit:** A dedicated hardware debounce circuit (RC filter or Schmitt trigger) would further stabilize keypad inputs.
- **LED Puzzle Feature:** Restoring the original LED-pattern dismissal challenge would significantly increase user engagement.
- **Motor Power:** An external motor driver IC (e.g. L293D) would ensure reliable motor operation independent of the microcontroller's current capacity.

This project demonstrated the value of iterative testing, flexibility in problem-solving, and striking a balance between technical viability and ambition. It provided practical experience with peripheral interfacing, microcontroller programming in Assembly, and debugging under real-world hardware constraints.